Reason. Plan. Act. Physical AI Workshop

Lab 2.2: Control

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



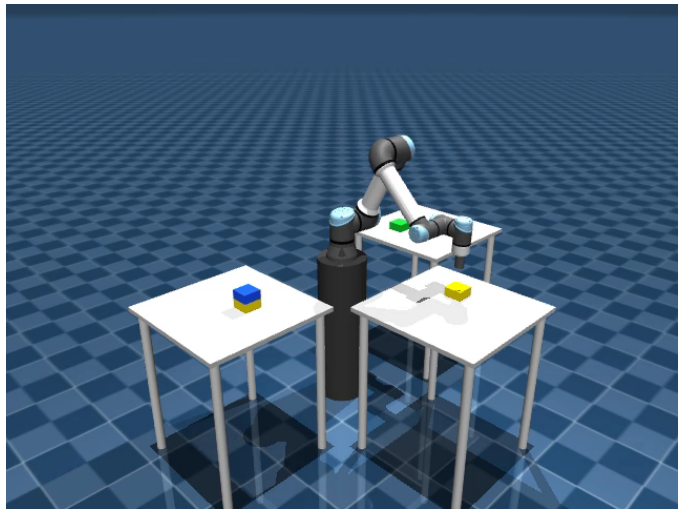
Where we are

Guy Azran and
Sarah Keren

In Lab 2.1 we planned collision-free paths and handed the waypoints to the robot's **position servos**.

We took the servos for granted:

- under gravity, the arm stops short of its target
- following a path, it lags behind and cuts corners



Today: what is inside a servo, and how to do better.

1.1 Task planning	2.1 Motion planning	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	**2.2 Control**	3.2 Decoupled TAMP	4.2 VLAs

Today

Learn	Actuators, torque motors, the control loop
Do	Task 1: a controller that never looks
Learn	Feedback: the P, I and D terms
Do	Task 2: P control
Do	Task 3: PD control
Do	Task 4: PID control
Do	Task 4.1: anti-windup
Do	Task 5: tune the gains by search
Learn	Feedforward: putting a model of the robot in the loop
Do	Task 6: gravity-compensated PD
Learn	Tracking a trajectory instead of a fixed target
Do	Task 7: from a path to a trajectory
Do	Task 7.1: use the velocity of the target
Do	Task 8 and bonus: execute the plan with our own controller

Inside the position servo

MuJoCo's general actuator applies a torque

$$\tau = \text{gain} \cdot u + b_0 + b_1\, q + b_2\, \dot{q}$$

With the UR5e's numbers (shoulder and elbow):

$$\tau = 2000\,(u - q) - 400\,\dot{q}$$

A **proportional-derivative controller**, hard-wired into the actuator, with torque limits of 150 Nm (28 Nm for the wrists).

Why does the arm sag? At rest, $2000\,(u - q)$ must balance the gravity torque τ_g, so the error is $\tau_g/2000$:

joint	gain × error	gravity torque
shoulder lift	30.9 Nm	30.9 Nm
elbow	12.0 Nm	12.0 Nm

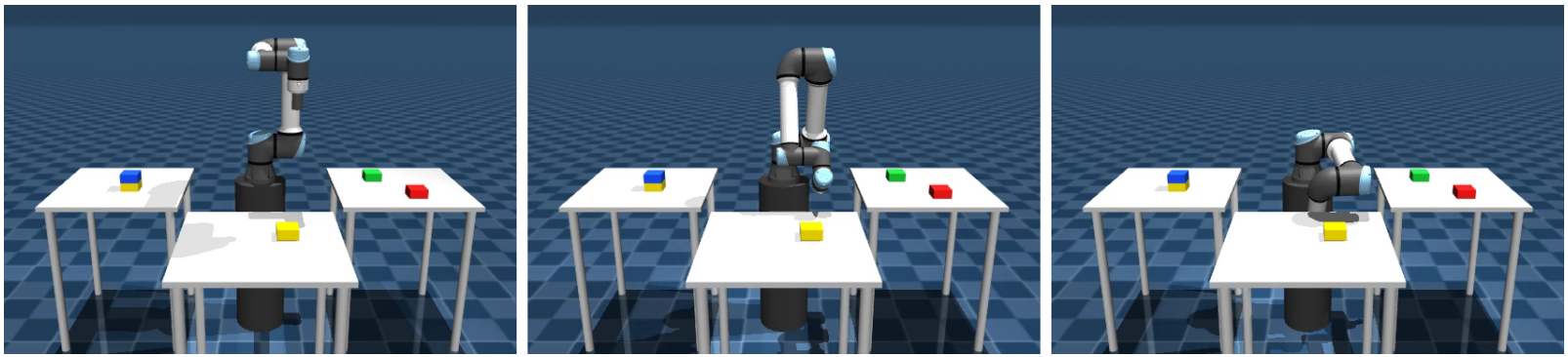
Motors: removing the controller

Guy Azran and Sarah Keren

A real motor does one thing: given a current, it applies a torque. Three numbers in the robot's specification make our actuators plain **torque motors**:

```
for actuator in robot.actuators:
    actuator.gainprm[0] = 1  # force = 1 * ctrl
    actuator.biasprm[:] = 0  #       + 0
    actuator.ctrlrange = actuator.forcerange  # ctrl is a torque
in Nm, within the limits of the motor
```

Nobody computes torques, so the arm collapses:



The control loop

Guy Azran and Sarah Keren

Once per time step, 500 times a second:

1. **measure** the joint angles q and velocities $\dot{q}$ (encoders, on a real robot)
2. **compute** the torques τ from the state and the target
3. **apply** the torques, and let the physics move on by one step

```
    for step in range(round(duration / model.opt.timestep)):
        t = data.time - start_time
        q, qdot, q_target = get_configuration(),
get_joint_velocities(), target_at(t)  # 1. measure
        tau = controller.torques(q, qdot, q_target)  # 2. compute
        data.ctrl[ctrl_idx] = tau  # 3. apply (MuJoCo clips the
torque to the motor's limits)
        mujoco.mj_step(model, data)
```

Every controller we write fills in step 2 only:
`Controller.torques(q, qdot, q_target)`.

Task 1: a controller that never looks

Guy Azran and
Sarah Keren

⚒ **Task 1**: Write `ConstantTorque`, a controller that ignores the state and always applies the same torque vector `tau`. Then use it to try and hold the arm at `HOME` with the torque that exactly cancels gravity there (`data.qfrc_bias[qvel_idx]`, read right after `reset()`): run it for three seconds starting from `HOME`, and then starting from `HOME + 0.05` (use `set_configuration`). Plot the two errors with `plot_errors` and watch the videos.

```python
class ConstantTorque(Controller):
    def __init__(self, tau):
        self.tau = np.asarray(tau, dtype=float)

    def torques(self, q, qdot, q_target):
        return self.tau  # your code here
```

- The torque that cancels gravity at `HOME` is known exactly. Does it hold the arm?
- What happens if the arm starts a little off?

Feedback: three ways to use the error

Measure the state, compare it with the target, act on the **error** $e(t) = q_{\text{target}} - q(t)$:

term	torque	behaves like
P	$K_p\, e(t)$	a spring pulling the joint to its target
D	$K_d\, \dot{e}(t)$	a damper resisting the motion
I	$K_i \int_0^t e(s)\, ds$	memory: pushes while the error persists

Together, the **PID controller**:

$$\tau(t) = K_p\, e(t) + K_i \int_0^t e(s)\, ds + K_d\, \dot{e}(t)$$

The most widely used controller: thermostats, cruise control, drones, and the joints of robot arms.

Task 2: P control

Guy Azran and
Sarah Keren

⚒ **Task 2**: Write the proportional controller `P`. Run it from `HOME` to `q_above_b3` for three seconds with K_p = 50, 200, 1000 and 5000, and plot the errors. Watch the video of one of the runs. Which gain would you choose?

```python
class P(Controller):
    def __init__(self, kp):
        self.kp = kp

    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

- $\tau = K_p\, e$: the stiffness of the spring, in Nm per radian
- Watch the video of one run, not only the plot

Task 3: PD control

Guy Azran and
Sarah Keren

⚒ **Task 3**: Write the `PD` controller. Run it from `HOME` to `q_above_b3` with $K_p = 1000$ and K_d = 0, 5, 20, 50 and 200, and plot the errors. Then look at the run with $K_d = 50$ joint by joint, with `plot_joints`: which joints reach their target, and which do not?

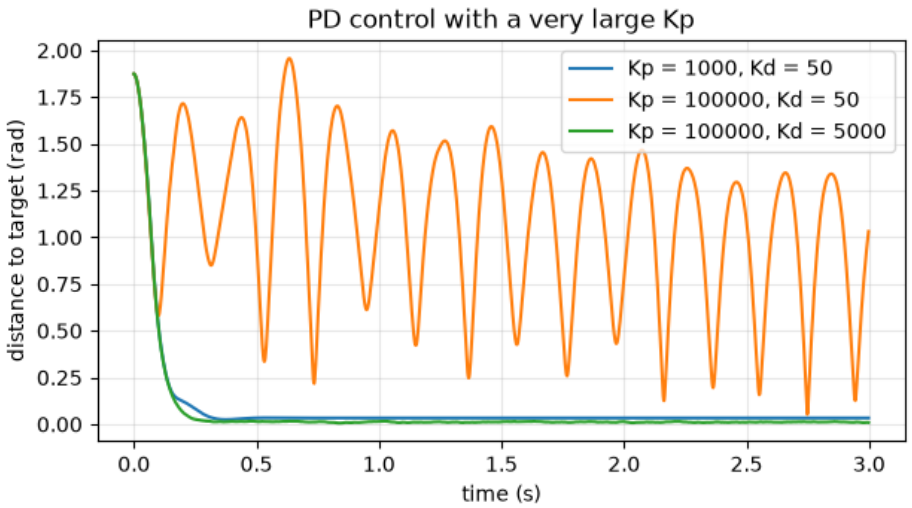
```
class PD(Controller):
    def __init__(self, kp, kd):
        self.kp = kp
        self.kd = kd

    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

- $\tau = K_p\, e + K_d\, \dot{e}$, and for a fixed target $\dot{e} = -\dot{q}$
- Compare the shapes of the curves: too little damping, too much

Can we just raise K_p?

Guy Azran and
Sarah Keren



The error at rest is τ_g / K_p.
So make K_p huge?

- tiny errors ask for huge torques: the motors saturate
- without much more damping, the arm rattles
- on a real robot, sensor noise is amplified too

Stiffness has a ceiling. We need another way to remove the residual error.

Task 4: PID control

Guy Azran and Sarah Keren

⚒ **Task 4**: Write the `PID` controller (you can build on `PD`). Run it from `HOME` to `q_above_b3` with $K_p = 1000$, $K_d = 50$ and K_i = 0, 200, 1000, 5000 and 20000, and plot the errors. What does the integral term improve, and what does it make worse?

```python
class PID(PD):
    def __init__(self, kp, kd, ki):
        super().__init__(kp, kd)
        self.ki = ki

    def reset(self):
        self.integral = np.zeros(6)

    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

- A running sum: add $e\,\Delta t$ at every step, clear it in `reset`

Task 4.1: anti-windup

Guy Azran and
Sarah Keren

⚒ **Task 4.1**: Add an `integral_limit` argument to `PID` (default: no limit) and clip the integral to $[-\text{limit}, \text{limit}]$ after every update. Compare the overshoot with $K_i = 5000$ without a limit and with a limit of 0.02, and try $K_i = 20000$ again with the limit.

```python
class PID(PD):
    def __init__(self, kp, kd, ki, integral_limit=np.inf):
        super().__init__(kp, kd)
        self.ki = ki
        self.integral_limit = integral_limit

    def reset(self):
        self.integral = np.zeros(6)

    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

- **Windup**: while a motor is saturated, the integral keeps growing, and later pushes the joint past its target

Task 5: tune the gains by search

Guy Azran and
Sarah Keren

⚒ **Task 5**: Search the grid of gains below (`itertools.product` enumerates all combinations) for the controller with the shortest settling time among those whose overshoot is below 0.1 rad. Print the best three. A run without video takes a fraction of a second, so the whole grid takes a few seconds.

```python
from itertools import product

KP_VALUES = [500, 1000, 2000, 4000]
KD_VALUES = [20, 50, 100, 200]
KI_VALUES = [0, 1000, 5000]

# your code here
```

- `evaluate`: settling time, overshoot, final error
- By hand: raise K_p, add K_d, then a little K_i

Are the best gains for this motion the best for every motion?

Knowing the robot: feedforward

The PID controller knows nothing about the robot: a number (the error) in, a number (the torque) out.

But we *do* know the robot. Its dynamic model gives the gravity torque in any configuration, $\tau_g(q)$ (in MuJoCo, `data.qfrc_bias`).

$$\tau(t) = \underbrace{\tau_g(q(t))}_{\text{feedforward}} + \underbrace{K_p\, e(t) + K_d\, \dot{e}(t)}_{\text{feedback}}$$

- the model holds the arm up, the feedback corrects what the model does not know
- the model is never complete: the block in the gripper, friction in the gearboxes, wear

Task 6: gravity compensation

Guy Azran and
Sarah Keren

⚒ **Task 6**: Write `GravityCompensatedPD`, a `PD` controller that adds the gravity torque `data.qfrc_bias[qvel_idx]` to its output (with the arm moving, `qfrc_bias` also contains small velocity-dependent terms; that is fine, they belong to the model too). Compare it with `PD(1000, 50)` and `PID(1000, 50, 5000, 0.02)`: plot the errors, and `evaluate` all three. Then try it with much softer gains, $K_p = 200$ and $K_d = 20$.

```python
class GravityCompensatedPD(PD):
    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

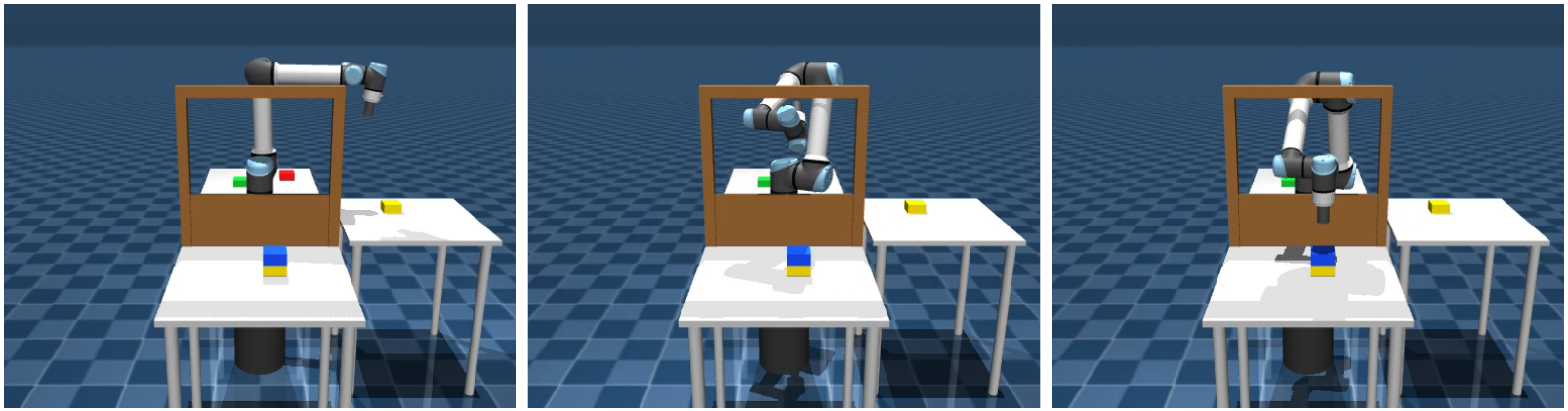
- Compare the settling time, the overshoot and the final error of all three
- What would softer gains be good for on a real robot?

Following a path

Guy Azran and Sarah Keren

In Lab 2.1, `follow_path` switched to the next waypoint when the robot came close enough. The robot never knew where it was going next.

A **trajectory** is a target that is a function of time, $q_{\text{target}}(t)$: we walk along the path at a constant speed, and the controller **tracks** the moving target.



From **HOME** to above **b2**, through the window, with torque motors

Task 7: from a path to a trajectory

Guy Azran and Sarah Keren

🛠️ **Task 7**: Write `trajectory_from_path(waypoints, speed)`. It returns two things: a function `target(t)` that gives the configuration on the path at time `t`, moving along it at `speed` radians per second (linear interpolation between waypoints; stay at the last waypoint after the end), and the duration of the trajectory. Hint: compute the cumulative distance along the path, `np.cumsum` and `np.searchsorted` do most of the work.

```
def trajectory_from_path(waypoints, speed):
    """A function of time that moves along the path at the given
speed (rad/s), and the duration of the motion."""
    waypoints = np.array(waypoints)

    def target(t):
        return waypoints[0]  # your code here

    duration = 1.0  # your code here
    return target, duration
```

- The cell then tracks the trajectory at 1 rad/s with `GravityCompensatedPD(1000, 50)`

Task 7.1: use the velocity of the target

Guy Azran and
Sarah Keren

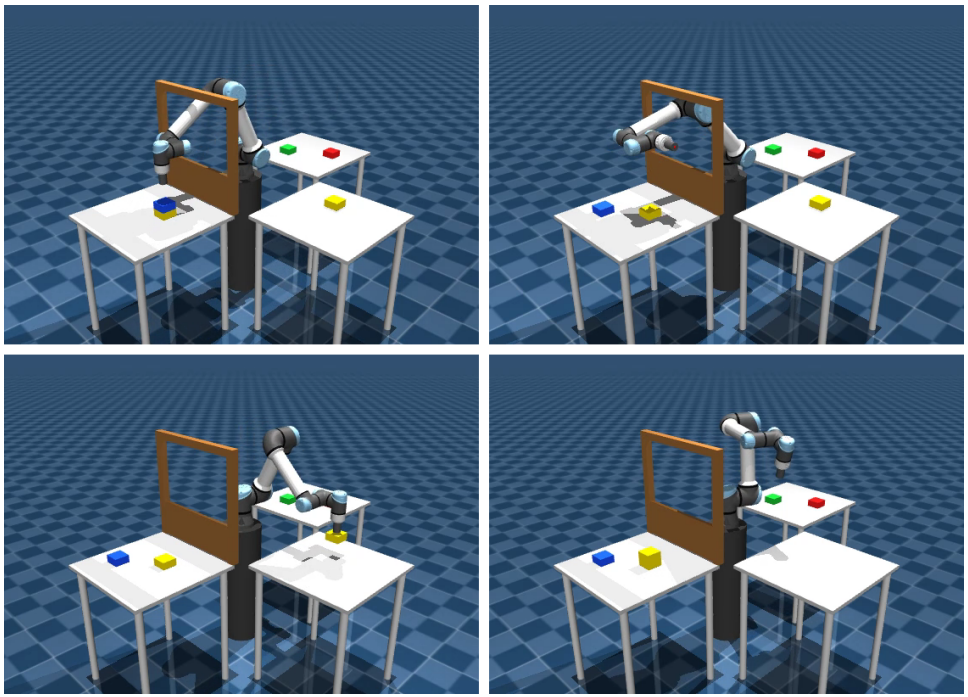
The PD controller trails a moving target: at speed v, $K_p e = K_d v$, so $e = K_d v / K_p$.
With a moving target, $\dot{e} = \dot{q}_{\text{target}} - \dot{q}$.

⚒ **Task 7.1**: Write `TrackingController`, a `GravityCompensatedPD` whose derivative term uses the velocity of the target, estimated from consecutive targets (remember to `reset` the memory of the previous target). Track the trajectory at 1 rad/s and compare the error with the previous run. Then compare both controllers, and the servo-equivalent `PD(2000, 400)`, at 1, 2, 4 and 8 rad/s: print the largest tracking error and the number of collision steps for each.

```python
class TrackingController(GravityCompensatedPD):
    def reset(self):
        self.previous_target = None

    def torques(self, q, qdot, q_target):
        return np.zeros(6)  # your code here
```

Executing the plan with our own controller



The plan of Lab 1.1, through the window:

- `unstack(b2, b1)`
- `put-down(b2, t1)`
- `pick-up(b3, t2)`
- `stack(b3, b1)`

Every motion is a trajectory tracked by our controller, then a short settling phase. With torque motors, the controller must never stop: between motions, the arm stays up only because we keep computing torques.

Task 8: replace the servos

Guy Azran and Sarah Keren

⚒ **Task 8**: Write `follow_path(waypoints, speed, tolerance, settling_time)` : build the trajectory, `run` the controller along it for the duration of the trajectory plus `settling_time` , and return whether the robot ended within `tolerance` of the last waypoint (and its video frames). `move_to(target)` is then a one-liner: a path with two waypoints, the current configuration and the target.

```
def follow_path(waypoints, speed=1.0, tolerance=0.01,
settling_time=1.0, camera="overview"):
    """Move the robot along the path with CONTROLLER. Returns
(success, frames)."""
    return True, []  # your code here


def move_to(target, speed=1.0, camera="overview"):
    """Move the robot in a straight line (in configuration space)
to the target configuration."""
    return True, []  # your code here
```

The cell then executes the plan, and reports the simulated time it took.

Bonus task: make it fast

Guy Azran and
Sarah Keren

⚒ **Bonus task**: Make it fast. Raise the speed of `follow_path` and `move_to` until something goes wrong, and find out what: does the block slip off the cup, does the arm hit the window, does the controller run out of torque? Then try to fix it with the gains, or with a different controller. (Note that `grasp` welds the block to the cup, so a real suction cup would let go long before ours does.)

```
# your code here
```

- The planner checked the *path* for collisions. Did anyone check the *trajectory*?
- Which failures can better gains fix, and which can they not?

Get to work!

Summary

In this tutorial we

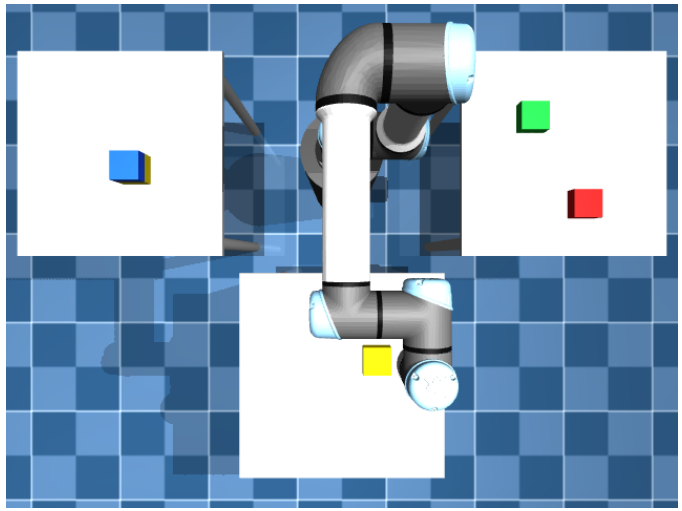
- found a PD controller inside the position servos, and replaced the servos with torque motors
- wrote the control loop, and saw open loop drift and feedback correct
- built PID term by term: P is a spring, D a damper, I a memory that needs anti-windup
- tuned the gains by search
- put a model of the robot in the loop: gravity compensation
- turned paths into trajectories, and tracked them using the target velocity
- executed the plan of Lab 1.1 with our own controller

What comes next

Beyond PID:

- **operational space control**: a force on the tool, $\tau = J^T F$
- **computed torque control**: the full equations of motion
- **model predictive control**: plan the torques over a horizon

On a real robot, angles come from encoders, velocities are differentiated, and masses are known to a few percent. The state has to be **estimated**.



Lab 3.1: state estimation, from a camera to facts